

# CS201 Midterm 3

15 May 2025

İlknur KARADENİZ and Olcay Taner YILDIZ

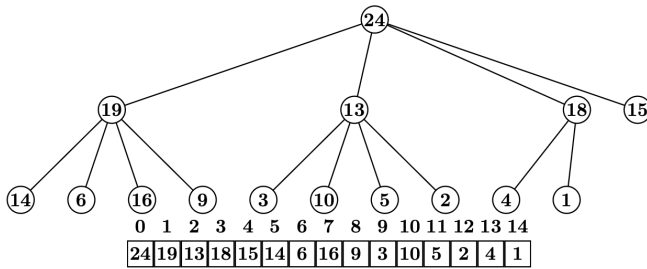
**Abstract—** Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures including arrays, (iv) you can not modify the input.

## I. QUESTION (HEAP)

Write a recursive method in **MaxDHeap** class

```
void descendants(int current,
               int* list,
               int& index)
```

which fills the descendants of the HeapNode with index *current* to the array list. Use and modify index to store the HeapNodes into correct positions. Your method should run in  $\mathcal{O}(N)$  time.



Descendants of 19 (with index 1) which are [14, 6, 16, 9] will be put into the list.

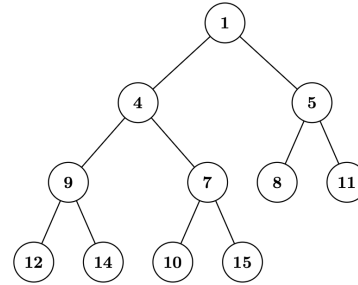
Descendants of 18 (with index 3) which are [4, 1] will be put into the list.

## II. QUESTION (HEAP)

Write a method in **MinHeap** class

```
int numbersPercolateDownToLeaves(int* list,
                                  int size)
```

that returns how many of the numbers in the list will percolate down to the leaves, if they start from the root of the heap tree. When percolating down in the tree, choose a random child each time. Do not modify the tree while checking. Size represents the size of the array list.



numbersPercolateDownToLeaves([3, 16, 7, 20], 4) will return 2. Only 16 and 20 will percolate down.

numbersPercolateDownToLeaves([1, 18], 4) will return 1. 18 will percolate down.

## III. QUESTION (DISJOINT SET)

In an unknown world, there are people (zombie) who bites other people (may or may not be zombie). One is converted to a zombie, if he/she was bitten by some other zombie. Given who bites whom in two arrays, identify the number of survivors, that is the number of people who hasn't been bitten yet. Write the static method

```
int numberOfSurvivors(int count,
                     int* whoBites,
                     int* whoWasBitten,
                     int size)
```

where whoBites[i] has bitten whoWasBitten[i] correspondingly. Count represents the number of people in the beginning. Size represents the size of the arrays whoBites and whoWasBitten. You are only allowed to use one external Disjoint Set.

numberOfSurvivors(6, [0, 2], [1, 5], 2) will return 2. 0 will bite 1, 2 will bite 5; only 3 and 4 are not bitten.

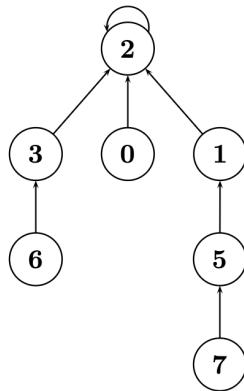
numberOfSurvivors(9, [0, 1, 2, 6], [1, 2, 4, 8], 2) will return 3. 0 will bite 1, 1 will bite 2, 2 will bite 4, 6 will bite 8; only 3, 5, 7 are not bitten.

#### IV. QUESTION (DISJOINT SET)

Given in the index of a set as *current*, write a recursive method

```
void descendants(int current,
               int* list,
               int& index)
```

that fills the descendants of that set to the array list. Use and modify index to store the indexes of the sets into correct positions.



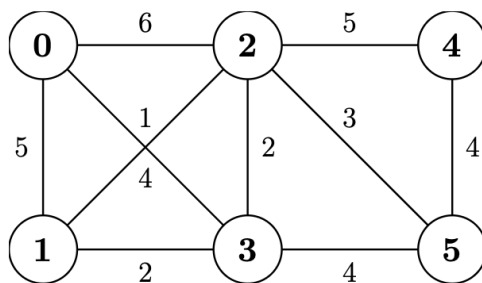
Descendants of 1 which are [5, 7] will be put into the list.  
 Descendants of 2 which are [3, 0, 1, 6, 5, 7] will be put into the list.

#### V. QUESTION (GRAPH)

Given a weighted undirected graph representing the distances between cities in a country, write a method in adjacency matrix representation

```
int capitalCity()
```

which identifies the index of the capital city. The capital city has the largest number of nearby cities compared to other cities. A city A is nearby to a city B, if its direct distance to city B is the smallest across all cities compared. Identify the number of nearby cities for every city, and use one array to store the number of nearby cities for every city.



0 is nearby to 3, because the distance between them (1) is the smallest compared to distances between cities 0 and 2, cities 0 and 1.

1 is nearby to 3, because the distance between them (2) is the smallest compared to distances between cities 1 and 0, cities 1 and 2.

2 is nearby to 3, because the distance between them (2) is the smallest compared to distances between cities 2 and 0, cities 2 and 1, cities 2 and 3, cities 2 and 4, cities 2 and 5.

3 is nearby to 0, because the distance between them (1) is the smallest compared to distances between cities 3 and 1, cities 3 and 1, cities 3 and 5.

4 is nearby to 5, because the distance between them (4) is the smallest compared to distances between cities 4 and 2.

5 is nearby to 2, because the distance between them (3) is the smallest compared to distances between cities 5 and 3, cities 5 and 4.

So, city 3 has the largest nearby cities (3), so it is a capital city.

#### VI. QUESTION (GRAPH)

Write a new constructor for Graph class for linked list implementation

```
Graph(Graph* graphs, int size)
```

which constructs a new graph by merging all graphs in the graph array. Size represents the number of graphs in graphs. You can assume that all graphs have the same size, and the intersections of the graphs are empty.